1. CREATE DATABASE
2. CONNECT DATABASE
3. CREATE TABLE
4. INSERT INTO
5. SELECT *
6. SELECT specific columns
7. ORDER BY
8. LIMIT
9. WHERE operators
   (> >= AND OR NOT BETWEEN LIKE IN IS NULL)
10. UPDATE
11. UPDATE + CASE WHEN ELSE
12. ALTER TABLE ADD COLUMN
13. ADD COLUMN from old column
14. ADD COLUMN using CASE
15. ALTER COLUMN TYPE
16. DELETE
17. DROP COLUMN
18. DROP TABLE

1. Database Commands

```
CREATE DATABASE company;

\c company;
```

2. Table Creation (Data Types + Constraints)

```
CREATE TABLE employees (
    employee_id SERIAL PRIMARY KEY,

    name VARCHAR(40) NOT NULL,

    hire_date DATE DEFAULT CURRENT_DATE,

    salary BIGINT CHECK (salary >= 0),

    description TEXT,

    emp_code CHAR(5) UNIQUE,

    birth_date DATE,

    login_time TIME,

    order_placed TIMESTAMP,

    seminar_schedule TIMESTAMPTZ,

    course_duration INTERVAL,

    active BOOLEAN DEFAULT TRUE,

    weight NUMERIC(6,3),
```

```
    age INTEGER CHECK(age >= 18),

    cgpa REAL,

    status VARCHAR(20) DEFAULT 'New',

    uuid UUID
);
```

3M

3. Insert Data

```
INSERT INTO employees
(name,salary,description,emp_code,birth_date,
 login_time,active,weight,age,cgpa,status)
VALUES
('Aman',50000,'Manager','A1234',
 '2001-01-15','09:00',TRUE,65.500,23,8.5,'Active'),

('Rahul',60000,'Developer','B1234',
 '2000-05-12','10:00',TRUE,70.200,24,8.2,'Active'),

('Priya',45000,'Analyst','C1234',
 '1999-08-10','08:30',FALSE,55.800,25,7.9,'Inactive');
```

3M

4. SELECT Queries

All Columns

```
SELECT *
FROM employees;
```

Selected Columns

```
SELECT name, salary
FROM employees;
```

Sorted Data

```
SELECT *
FROM employees
ORDER BY salary DESC;
```

Ascending + Limit 5

```
SELECT name, salary
FROM employees
ORDER BY salary ASC
LIMIT 5;
```

3M

5. WHERE Operators Together

Greater Than

```
SELECT *
FROM employees
WHERE salary > 50000;
```

Greater Than Equal To

```
SELECT *
FROM employees
WHERE salary >= 50000;
```

AND

```
SELECT *
FROM employees
WHERE age > 22
AND salary > 40000;
```

OR

```
SELECT *
FROM employees
WHERE department='IT'
OR salary > 50000;
```

NOT

```
SELECT *
FROM employees
WHERE NOT status='Inactive';
```

BETWEEN

```
SELECT *
FROM employees
WHERE salary BETWEEN 40000 AND 60000;
```

LIKE

```
SELECT *
FROM employees
WHERE name LIKE 'A%';
```

IN

```
SELECT *
FROM employees
WHERE age IN (23,24,25);
```

IS NULL

```
SELECT *
FROM employees
```

```
WHERE uuid IS NULL;
```

3M

6. UPDATE Queries Together

Basic Update

```
UPDATE employees
SET salary = 55000
WHERE name='Aman';
```

Greater Than

```
UPDATE employees
SET status='Senior'
WHERE salary > 55000;
```

LIKE

```
UPDATE employees
SET description='Team Lead'
WHERE name LIKE 'R%';
```

AND

```
UPDATE employees
SET salary = salary + 5000
WHERE age > 22
AND status='Active';
```

BETWEEN

```
UPDATE employees
SET bonus = 5000
WHERE salary BETWEEN 45000 AND 60000;
```

3M

7. CASE WHEN ELSE END

Update Existing Column

```
UPDATE employees
SET status =
CASE
    WHEN salary >= 60000 THEN 'High'
    WHEN salary >= 50000 THEN 'Medium'
    ELSE 'Low'
END;
```

3M

8. Add New Columns

Simple Column

```
ALTER TABLE employees
ADD COLUMN email VARCHAR(100);
```

Add Column with Constraint

```
ALTER TABLE employees
ADD COLUMN joining_date DATE NOT NULL
DEFAULT CURRENT_DATE;
```

3M

9. Add Column Based on Old Column

```
ALTER TABLE employees
ADD COLUMN annual_salary NUMERIC(12,2);

UPDATE employees
SET annual_salary = salary * 12;
```

3M

10. Add Column Using CASE

```
ALTER TABLE employees
ADD COLUMN grade VARCHAR(10);

UPDATE employees
SET grade =
CASE
    WHEN salary >= 60000 THEN 'A'
    WHEN salary >= 50000 THEN 'B'
    ELSE 'C'
END;
```

3M

11. CASE with Operators

```
UPDATE employees
SET grade =
CASE
    WHEN age >= 25 AND salary > 55000 THEN 'A'
    WHEN age BETWEEN 22 AND 24 THEN 'B'
    WHEN name LIKE 'A%' THEN 'C'
    ELSE 'D'
END;
```

3M

12. Change Data Type

```
ALTER TABLE employees
ALTER COLUMN age TYPE SMALLINT;

ALTER TABLE employees
ALTER COLUMN name TYPE VARCHAR(100);
```

3M

13. Optional Table Operations

Rename Column

```
ALTER TABLE employees
RENAME COLUMN hire_date TO joining_date;
```

Drop Column

```
ALTER TABLE employees
DROP COLUMN cgpa;
```

Delete Row

```
DELETE FROM employees
WHERE employee_id = 1;
```

Drop Table

```
DROP TABLE employees;
```

3M

PostgreSQL Practical Flow (With Meaning)

3M

1. CREATE DATABASE

Purpose:
Creates a new database.

Syntax:

```
CREATE DATABASE database_name;
```

Example:

```
CREATE DATABASE company;
```

Connect to database:

```
\c company;
```

3M

2. CREATE TABLE

Purpose:
Creates a table and defines columns, data types and constraints.

Syntax:

```
CREATE TABLE table_name(
    column datatype constraint
);
```

Example:

```
CREATE TABLE employees(
    emp_id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    department VARCHAR(30),
    salary NUMERIC(10,2),
    age INT,
    email VARCHAR(100),
    hire_date DATE
);
```

Meaning:

SERIAL → Auto increment number

PRIMARY KEY → Unique identifier

VARCHAR(50) → Text up to 50 characters

NUMERIC(10,2) → Decimal value

INT → Integer value

DATE → Date only

3. IMPORT CSV

Purpose:
Imports CSV file data into SQL table.

CSV Example:

```
emp_id,name,department,salary,age
1,Aman,IT,50000,23
2,Rahul,HR,60000,24
```

SQL Method:

```
COPY employees(emp_id,name,department,salary,age)
FROM '/path/employees.csv'
DELIMITER ','
CSV HEADER;
```

Meaning:

COPY → Import file data

DELIMITER ‘, ’→ Columns separated by comma

CSV → File format

HEADER → First row contains column names

3M

pgAdmin Method

Purpose:
Import CSV without SQL command.

Steps:

1. Right Click Table
2. Import / Export Data
3. Select CSV File
4. Format = CSV
5. Header = YES
6. Delimiter = ,
7. Click OK

3M

4. SELECT

Purpose:
Retrieve data from table.

All columns:

```
SELECT *
FROM employees;
```

Meaning:

- = All columns

FROM = Source table

3M

Selected columns:

```
SELECT name,salary
FROM employees;
```

Meaning:

Returns only name and salary.

3M

5. WHERE OPERATORS

Purpose:
Filter rows based on conditions.

Salary greater than 50000:

```
SELECT *
FROM employees
WHERE salary > 50000;
```

Meaning:

Returns rows having salary more than 50000.

3M

Greater than equal:

```
WHERE salary >= 50000
```

Meaning:

Salary 50000 or more.

3M

AND

```
WHERE age > 20
AND salary > 40000
```

Meaning:

Both conditions must be true.

3M

OR

```
WHERE age > 20
OR salary > 40000
```

Meaning:

Any one condition can be true.

3M

NOT

```
WHERE NOT department='IT'
```

Meaning:

Returns all departments except IT.

3M

BETWEEN

```
WHERE salary BETWEEN 40000 AND 70000
```

Meaning:

Range including both values.

3M

LIKE

```
WHERE name LIKE 'A%'
```

Meaning:

Starts with A.

Examples:

```
'A%'  → Starts with A

'%A'  → Ends with A

'%A%' → Contains A
```

3M

IN

```
WHERE department IN ('IT','HR')
```

Meaning:

Equivalent to:

```
department='IT'
OR department='HR'
```

3M

IS NULL

```
WHERE email IS NULL
```

Meaning:

Returns rows where email is missing.

3M

6. UNION OPERATORS

Purpose:
Combine results of multiple SELECT statements.

UNION

```
SELECT name
FROM student1

UNION

SELECT name
FROM student2;
```

Meaning:

Combines results and removes duplicates.

3M

UNION ALL

```
SELECT name
FROM student1

UNION ALL

SELECT name
FROM student2;
```

Meaning:

Combines results and keeps duplicates.

3M

INTERSECT

```
SELECT name
FROM student1

INTERSECT

SELECT name
FROM student2;
```

Meaning:

Only common rows.

3M

EXCEPT

```
SELECT name
FROM student1

EXCEPT

SELECT name
FROM student2;
```

Meaning:

Rows present in first query only.

3M

7. AGGREGATE FUNCTIONS

Purpose:
Perform calculations on multiple rows.

SUM()

```
SELECT SUM(salary)
AS total_salary
FROM employees;
```

Meaning:

Adds all salary values.

3M

SUM with WHERE

```
SELECT SUM(salary)
FROM employees
WHERE department='IT';
```

Meaning:

Adds salaries of IT employees only.

3M

AVG()

```
SELECT AVG(salary)
FROM employees;
```

Meaning:

Average salary.

3M

MAX()

```
SELECT MAX(salary)
FROM employees;
```

Meaning:

Highest salary.

3M

MIN()

```
SELECT MIN(salary)
FROM employees;
```

Meaning:

Lowest salary.

3M

COUNT()

```
SELECT COUNT()
FROM employees;
```

Meaning:

Counts total rows.

3M

COUNT with LIKE

```
SELECT COUNT()
FROM employees
WHERE name LIKE 'A%';
```

Meaning:

Counts employees whose names start with A.

3M

AVG with OR

```
SELECT AVG(salary)
FROM employees
WHERE age>30
OR department='IT';
```

Meaning:

Average salary of employees satisfying either condition.

3M

8. STRING FUNCTIONS

Purpose:
Manipulate text data.

UPPER()

```
SELECT UPPER(category)
AS cat1
FROM products;
```

Meaning:

Converts text to CAPITAL letters.

3M

LOWER()

```
SELECT LOWER(category)
AS cat2
FROM products;
```

Meaning:

Converts text to lowercase.

3M

CONCAT()

```
SELECT CONCAT(first_name,' ',last_name)
AS full_name
FROM employees;
```

Meaning:

Joins strings together.

3M

LENGTH()

```
SELECT LENGTH(name)
FROM employees;
```

Meaning:

Returns number of characters.

3M

SUBSTRING()

```
SELECT SUBSTRING(name,2,5)
FROM employees;
```

Meaning:

Start from position 2 and return next 5 characters.

3M

LEFT()

```
SELECT LEFT(name,3)
FROM employees;
```

Meaning:

Returns first 3 characters.

3M

RIGHT()

```
SELECT RIGHT(name,3)
FROM employees;
```

Meaning:

Returns last 3 characters.

3M

TRIM()

```
SELECT TRIM('   Aman   ');
```

Meaning:

Removes extra spaces.

Result:

```
Aman
```

3M

9. DATE FUNCTIONS

NOW()

```
SELECT NOW();
```

Meaning:

Current date and current time.

3M

CURRENT_DATE

```
SELECT CURRENT_DATE;
```

Meaning:

Today’s date only.

3M

EXTRACT()

```
SELECT EXTRACT(YEAR FROM hire_date)
FROM employees;
```

Meaning:

Extract year.

```
SELECT EXTRACT(MONTH FROM hire_date);
```

Extract month.

```
SELECT EXTRACT(DAY FROM hire_date);
```

Extract day.

3M

10. STRING TO DATE

Purpose:
Convert text into date.

```
SELECT TO_DATE(
'25-12-2025',
'DD-MM-YYYY'
);
```

Meaning:

Text becomes date datatype.

3M

11. TO_CHAR

Purpose:
Convert date to formatted text.

```
SELECT TO_CHAR(hire_date,'DD-MM-YYYY')
FROM employees;
```

Meaning:

Displays date as 25-12-2025.

3M

Month Name

```
SELECT TO_CHAR(hire_date,'Month');
```

Example:

December

3M

Day Name

```
SELECT TO_CHAR(hire_date,'Day');
```

Example:

Monday

3M

Year

```
SELECT TO_CHAR(hire_date,'YYYY');
```

Example:

2025

3M

12. DATE_PART

Purpose:
Extract specific part of date.

```
SELECT DATE_PART('year',hire_date);

SELECT DATE_PART('month',hire_date);

SELECT DATE_PART('day',hire_date);
```

Meaning:

Returns only requested component.

3M

13. DATE_TRUNC

Purpose:
Rounds date to beginning of unit.

Month Start

```
SELECT DATE_TRUNC('month',NOW());
```

Meaning:

First day of current month.

3M

Year Start

```
SELECT DATE_TRUNC('year',NOW());
```

Meaning:

1 January of current year.

3M

14. COALESCE

Purpose:
Replace NULL values.

```
SELECT name,
COALESCE(bonus,0)
FROM employees;
```

Meaning:

If bonus is NULL use 0.

3M

15. CASE

Purpose:
Create new values based on conditions.

Syntax:

```
CASE
WHEN condition THEN value
WHEN condition THEN value
ELSE value
END
```

3M

CASE Example

```
SELECT name,
salary,

CASE
WHEN salary>=70000 THEN 'High'
WHEN salary>=50000 THEN 'Medium'
ELSE 'Low'
END AS salary_grade

FROM employees;
```

Meaning:

Creates new column salary_grade.

3M

CASE with IN

```
CASE
WHEN department IN ('IT','HR')
THEN 'Office'
ELSE 'Other'
END
```

3M

CASE with BETWEEN

```
CASE
WHEN salary BETWEEN 50000 AND 70000
THEN 'Mid'
ELSE 'Other'
END
```

3M

UPDATE USING CASE

```
UPDATE employees
```

```
SET salary_grade =

CASE
WHEN salary>=70000 THEN 'A'
WHEN salary>=50000 THEN 'B'
ELSE 'C'
END;
```

Meaning:

Updates existing rows based on salary.

3M

16. WINDOW FUNCTIONS

Sample Data

Product	Category	Price
A	Mobile	200
B	Mobile	400
C	Mobile	600
D	Laptop	500

3M

ROW_NUMBER()

```
SELECT *,
ROW_NUMBER()
OVER(PARTITION BY category
ORDER BY price DESC)
FROM products;
```

Meaning:

Unique numbering within category.

3M

RANK()

```
SELECT *,
RANK()
OVER(ORDER BY price DESC)
FROM products;
```

Meaning:

Same rank for ties.
Gaps appear.

3M

DENSE_RANK()

```
SELECT *,
DENSE_RANK()
OVER(ORDER BY price DESC)
FROM products;
```

Meaning:

Same rank for ties.
No gaps.

3️⃣

Running SUM

```
SELECT *,
SUM(price)
OVER(ORDER BY price)
FROM products;
```

Meaning:

Cumulative total.

3️⃣

Running AVG

```
SELECT *,
AVG(price)
OVER(ORDER BY price)
FROM products;
```

Meaning:

Running average.

3️⃣

17. JOINS

Purpose:
Combine data from multiple tables.

INNER JOIN

```
SELECT *
FROM employees e
INNER JOIN departments d
ON e.dept_id=d.dept_id;
```

Meaning:

Only matching rows.

3️⃣

LEFT JOIN

```
SELECT *
FROM employees e
LEFT JOIN departments d
ON e.dept_id=d.dept_id;
```

Meaning:

All employees.
Matching departments only.

3️⃣

RIGHT JOIN

```
SELECT *
FROM employees e
RIGHT JOIN departments d
ON e.dept_id=d.dept_id;
```

Meaning:

All departments.
Matching employees only.

3️⃣

FULL JOIN

```
SELECT *
FROM employees e
FULL JOIN departments d
ON e.dept_id=d.dept_id;
```

Meaning:

All rows from both tables.

3️⃣

CROSS JOIN

```
SELECT *
FROM employees
CROSS JOIN departments;
```

Meaning:

Every employee paired with every department.

3️⃣

COMPLETE PRACTICAL ORDER

1. CREATE DATABASE
2. CREATE TABLE
3. IMPORT CSV
4. INSERT
5. SELECT
6. WHERE
7. ORDER BY
8. LIMIT
9. AGGREGATE FUNCTIONS
10. UNION / UNION ALL / INTERSECT / EXCEPT
11. STRING FUNCTIONS
12. DATE FUNCTIONS
13. CASE
14. COALESCE
15. UPDATE
16. ALTER TABLE
17. WINDOW FUNCTIONS
18. JOINS
19. DELETE
20. DROP TABLE

mple Data

Name	Gender	Salary	Dept
Aman	M	80000	IT
Rohit	M	60000	HR
Priya	F	90000	Finance
Neha	F	75000	IT
Ankit	M	80000	IT



1. GROUP BY

```
SELECT Dept,
       COUNT(*) AS Total_Emp,
       SUM(Salary) AS Total_Salary
```

```
FROM Employees
GROUP BY Dept;
```

Output

Dept	Total_Emp	Total_Salary
IT	3	235000
HR	1	60000
Finance	1	90000

GROUP BY: Creates groups and performs aggregate functions on each group.

3️⃣

2. GROUP BY + WHERE + HAVING

```
SELECT Dept,
       SUM(Salary) AS Total_Salary
FROM Employees
WHERE Salary BETWEEN 60000 AND 90000
GROUP BY Dept
HAVING SUM(Salary) > 100000
ORDER BY Total_Salary DESC
LIMIT 5;
```

Output

Dept	Total_Salary
IT	235000

Order of Execution

```
FROM
↓
WHERE
↓
GROUP BY
↓
HAVING
↓
SELECT
↓
ORDER BY
↓
LIMIT
```

WHERE: Filters rows before grouping.

HAVING: Filters groups after grouping.

3️⃣

3. ROW_NUMBER()

```
SELECT Name,
       Salary,
       ROW_NUMBER() OVER
       (ORDER BY Salary DESC) AS Row_No
FROM Employees;
```

Output

Name	Salary	Row_No
Priya	90000	1
Aman	80000	2
Ankit	80000	3
Neha	75000	4
Rohit	60000	5

Pattern: 1,2,3,4,5

3M

4. RANK()

```
SELECT Name,
       Salary,
       RANK() OVER
       (ORDER BY Salary DESC) AS Rank_No
FROM Employees;
```

Output

Name	Salary	Rank
Priya	90000	1
Aman	80000	2
Ankit	80000	2
Neha	75000	4
Rohit	60000	5

Pattern: 1,2,2,4,5

3M

5. DENSE_RANK()

```
SELECT Name,
       Salary,
       DENSE_RANK() OVER
       (ORDER BY Salary DESC) AS Dense_Rank
FROM Employees;
```

Output

Name	Salary	Dense_Rank
Priya	90000	1
Aman	80000	2
Ankit	80000	2
Neha	75000	3
Rohit	60000	4

Pattern: 1,2,2,3,4

Quick Revision

Function	Duplicate Values
ROW_NUMBER()	Different numbers
RANK()	Same rank, skip next
DENSE_RANK()	Same rank, no skip

Examples:

```
ROW_NUMBER : 1,2,3
RANK       : 1,1,3
DENSE_RANK : 1,1,2
```

This is usually enough for PostgreSQL viva/interview/exam notes in 1–2 pages.